kernel after the basic system has been set up. All processes are children of the *init* process. When the system is shut down, init is the last process to terminate before the kernel executes its own shutdown.

To view the security contexts allotted to all system processes, just execute the pstree command with the Z option:

```
[root@vbg-work ~]# pstree -Z
init(`system_u:system_r:init_t:s0')
 |-NetworkManager(`system_u:system_r:NetworkManager_t:s0')
 |  |-dhclient(`system_u:system_r:dhcpc_t:s0')
 |  `-{NetworkManager}(`system_u:system_r:NetworkManager_t:s0')
 |-abrtd(`system_u:system_r:abrt_t:s0-s0:c0.c1023')
 |-acpid(`system_u:system_r:apmd_t:s0')
 |-atd(`system_u:system_r:crond_t:s0-s0:c0.c1023')
 |-auditd(`system_u:system_r:auditd_t:s0')
 |  |-audispd(`system_u:system_r:audisp_t:s0')
 |  |  |-sedispatch(`system_u:system_r:audisp_t:s0')
 |  |  `-{audispd}(`system_u:system_r:audisp_t:s0')
 |  `-{auditd}(`system_u:system_r:auditd_t:s0')
 |-avahi-daemon(`system_u:system_r:avahi_t:s0')
 |  `-avahi-daemon(`system_u:system_r:avahi_t:s0')
 |-bonobo-activati(`unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023')
 |  `-{bonobo-activat}(`unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023')
 |-cinserver(`system_u:system_r:initrc_t:s0')
 |  |-{cinserver}(`system_u:system_r:initrc_t:s0')
 |  |-{cinserver}(`system_u:system_r:initrc_t:s0')
 |  `-{cinserver}(`system_u:system_r:initrc_t:s0')
 |-clock-applet(`unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023')
 |-console-kit-dae(`system_u:system_r:consolekit_t:s0-s0:c0.c1023')
 |  |-{console-kit-da}(`system_u:system_r:consolekit_t:s0-s0:c0.c1023')
 |  |-{console-kit-da}(`system_u:system_r:consolekit_t:s0-s0:c0.c1023')
 |  |-{console-kit-da}(`system_u:system_r:consolekit_t:s0-s0:c0.c1023')
 |  |-{console-kit-da}(`system_u:system_r:consolekit_t:s0-s0:c0.c1023')
 |  |-{console-kit-da}(`system_u:system_r:consolekit_t:s0-s0:c0.c1023')
 |  |-{console-kit-da}(`system_u:system_r:consolekit_t:s0-s0:c0.c1023')
 |  |-{console-kit-da}(`system_u:system_r:consolekit_t:s0-s0:c0.c1023')
 |  |-{console-kit-da}(`system_u:system_r:consolekit_t:s0-s0:c0.c1023')
 |  |-{console-kit-da}(`system_u:system_r:consolekit_t:s0-s0:c0.c1023')
 |  |-{console-kit-da}(`system_u:system_r:consolekit_t:s0-s0:c0.c1023')
 |  |-{console-kit-da}(`system_u:system_r:consolekit_t:s0-s0:c0.c1023')
 |  |-{console-kit-da}(`system_u:system_r:consolekit_t:s0-s0:c0.c1023')
 |  |-{console-kit-da}(`system_u:system_r:consolekit_t:s0-s0:c0.c1023')
 |  |-{console-kit-da}(`system_u:system_r:consolekit_t:s0-s0:c0.c1023')
```

Now, we can see the security contexts associated with all the processes. How did the init process get a type of init_t? Why are the security contexts of some child processes different from those of the parent (else all processes in the system should have had the type *init_t*)? How are these different security types assigned?

Consider another scenario: The *httpd* process (Apache Web server) on a Fedora/RHEL server is started with the command *service httpd start* or */etc/init.d/httpd start*. Start the Web server process, if it's not already running, with the following command:

```
[root@vbg-work ~]# /etc/init.d/httpd start
```

Check the security context associated with the httpd process:

```
[root@vbg-work ~]# ps axZ | grep httpd
unconfined_u:system_r:httpd_t:s0 3099 ?        Ss     0:00 /usr/sbin/httpd
unconfined_u:system_r:httpd_t:s0 3102 ?        S      0:00 /usr/sbin/httpd
unconfined_u:system_r:httpd_t:s0 3103 ?        S      0:00 /usr/sbin/httpd
unconfined_u:system_r:httpd_t:s0 3104 ?        S      0:00 /usr/sbin/httpd
unconfined_u:system_r:httpd_t:s0 3105 ?        S      0:00 /usr/sbin/httpd
unconfined_u:system_r:httpd_t:s0 3106 ?        S      0:00 /usr/sbin/httpd
unconfined_u:system_r:httpd_t:s0 3107 ?        S      0:00 /usr/sbin/httpd
unconfined_u:system_r:httpd_t:s0 3108 ?        S      0:00 /usr/sbin/httpd
unconfined_u:system_r:httpd_t:s0 3109 ?        S      0:00 /usr/sbin/httpd
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 3113 pts/0 S+   0:00 grep httpd
```

As you can see, the httpd process gets a security context type of httpd_t. Stop the Web server with the following command:

```
[root@vbg-work ~]# service httpd stop
Stopping httpd:                                            [  OK  ]
```

Restart the Web server. But this time, not from the service start-up script, but by executing the binary command, which is as follows:

```
[root@vbg-work ~]# /usr/sbin/httpd
```

...and now, check the SELinux security context for the Web server process:

```
[root@vbg-work ~]# ps axZ | grep httpd
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 3207 ? Ss   0:00 /usr/sbin/httpd
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 3209 ? S    0:00 /usr/sbin/httpd
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 3210 ? S    0:00 /usr/sbin/httpd
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 3211 ? S    0:00 /usr/sbin/httpd
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 3212 ? S    0:00 /usr/sbin/httpd
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 3213 ? S    0:00 /usr/sbin/httpd
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 3214 ? S    0:00 /usr/sbin/httpd
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 3215 ? S    0:00 /usr/sbin/httpd
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 3216 ? S    0:00 /usr/sbin/httpd
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 3218 pts/0 S+   0:00 grep httpd
[root@vbg-work ~]#
```

This time, the type of the Web server process is *unconfined_t*. Why is it so? If we use the service command, the same Web server process gets a security context type of *httpd_t*, but executing the Web server directly gives the Web server process a security context type of *unconfined_t*.

As you would have probably guessed by now, this is due to type transitioning. Like objects (files), type transitioning is also applied to subjects (processes).

The basic rules to set the security context of services are the same as those for files: Whenever a new service (process) is created in the memory, type transition rules in the policy are checked. If a type transition rule exists and matches the